Scaling of the plane-quartic index calculus, q = 5 .. 23 (lead L5, 2026-10-18)

Status: toy-size measurement on the existing `quartic_ic.rs`; no sub-rho claim; no change to README.md, PROTOCOL.md or earlier results (new files only, plus counters added to `quartic_ic.rs`, see below).

Question

Does the recorded `quartic_ic` configuration reproduce, and how does its counted cost scale with ℓ for `q > 11`, against the matched rho on the order-ℓ subgroup of `E0(F_{q³})`? Diem's `Õ(q^{4/3})` with $\ell \approx q^3/4$ is slope 4/9 in `log2 ℓ`; rho is slope 1/2.

Commands (host: 4 cores, 15 GB, shared, load average 9-13, no isolation)

```
cd research/ghs_cubic_quartic_ic_20261012
rustc --edition 2021 -O quartic_ic.rs        # rustc 1.93.1
./quartic_ic 7 10 full                       # step 1, reproduction
./quartic_ic <q> <N> full --counters         # N = 10 for q <= 13, N = 1 for q = 17,
19, 23
./quartic_ic <q> 10 rho --counters           # q = 5 .. 23, 29, 31, 37, 41, 43
reports/scaling-20261018/analyze.py results reports/scaling-20261018   # fit + SVG
```

Files: `results/run-20261018T1600Z-reproduce.jsonl` (step 1), `results/run-20261018-scaling-q<q>.jsonl` (raw records per q), `results/run-20261018-canon3-collisions-q5-13.jsonl`, `reports/scaling-20261018/{analyze.py,scaling.json,cost_vs_log2ell.svg}`.

Step 1: reproduction

`./quartic_ic 7 10 full`: 10/10 `correct:true`, `recovery:ic`; the ten trial lines (k, `ops` = 25 959 133, `S` = 2 751 662.594680) are byte-identical to the q = 7 `full` lines of `run-20261014T0241Z.jsonl`. Mean rho steps for q = 5, 7, 11 (9.0, 11.5, 30.7) also match the recorded values.

Counters added (new; no algorithm changed)

Argument `--counters` prints extra records: per-phase differences of the tool's existing `Cost::total()` (pinned units of PROTOCOL.md: F_q mult 1, K mult 9, K inversion 90, ...), the number of factor-base pairs tried, distinct 4-point line relations, matrix width, valid transfers, degree-6 keys and degree-3 collisions, and an instance record (q, ℓ, #E0(K), #C(F_q)). A diagnostic Gaussian elimination mod ℓ of the relation rows (rank and row-op cost, mult 9 / add 3 / inv 90) is run only under `--counters`; it is not part of the recovery path and is not included in `ops_total`. With the counters off the output is unchanged (q = 5, 7 `ops` verified equal to the recorded run). Without them the tool prints only `ops`; its rho arm prints `ops: 0` because E0 arithmetic on the order-ℓ subgroup is a table lookup (the table is built in `E0::build`), so rho is priced here by formula, below.

Instances (first split instance per q, as in the recorded study)

The construction (4 | #E0(K), ℓ = #E0(K)/4 prime, split `u`) exists for every q tried, q = 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43. `#C(F_q) = q + 1` in all of them. `#Jac(C)(F_q)` is taken as `#E0(K)` (the tool's `ord`; Jac(C) is isogenous to the Weil restriction); it is not recomputed independently here.

Raw numbers

Unit: PROTOCOL pinned units. IC = `ops_total` (cold, public inputs to returned scalar, identical in every target of a given q: all trials at one q show one `ops` value). Matched rho = `√(πℓ/4)` steps (negation) × 116 (K affine add 108 = 2 mult + inversion, + 4 lookup + 4 canonicalization, PROTOCOL pins). "rho meas." = mean of the tool's own plain-rho step counts (10 targets) × 112.

q	ℓ	#E0(K)	#C(F_q)	targets, correct	IC ops	rho √(πℓ/4) steps	rho cost	IC / rho	rho meas. steps (mean)	IC / rho meas.
5	37	148	6	10, 10	3 959 246	5.39	625	6.3e3	9.0	3.9e3
7	89	356	8	10, 10	25 959 133	8.36	970	2.7e4	11.5	2.0e4
11	331	1324	12	10, 10	689 897 351	16.1	1 870	3.7e5	30.7	2.0e5
13	557	2228	14	10, 10	2 872 411 940	20.9	2 426	1.2e6	24.2	1.1e6
17	1231	4924	18	1, 1	19 577 784 383	31.1	3 607	5.4e6	56.1	3.1e6
19	1733	6932	20	1, 1	53 297 594 319	36.9	4 280	1.2e7	47.0	1.0e7
23	3037	12148	24	1, 1	212 286 177 920	48.8	5 665	3.7e7	88.7	2.1e7

Instances only (full IC not run): q = 29 (ℓ 6067), 31 (7459), 37 (12637), 41 (17257), 43 (19973); rho 10/10 correct at each (steps in the raw files).

Phase split of IC ops (from `--counters`; "C(K) enum" = the `enum_curve_k` scan of all q^6 affine K-points, run twice per trial; "φ table" = E0 build plus `transfer_div`/`pic_canon` for every n in 1..ℓ-1, twice):

q	C(K) enum (2×)	line relations	φ(nG) table	target	pairs tried	relations	columns	rank mod ℓ	elim. row ops	valid transfers	deg-6 keys (collisions)
5	281 250	525	3 667 779	2 754	15	0	7	0	0	24	21 (0)
7	2 117 682	1 316	23 823 657	3 024	28	1	9	1	135	44	41 (0)
11	31 888 098	4 686	657 959 904	3 348	66	3	13	3	549	164	164 (0)

13	86 882 562	7 553	2 785 451 859	3 402	91	4	15	4	954	296	296 (0)
17	434 476 242	16 371	19 143 146 763	3 672	153	8	19	8	2 814	596	589 (0)
19	846 825 858	22 610	52 450 546 068	3 726	190	12	21	12	4 887	924	924 (0)
23	2 664 646 002	39 468	209 621 149 263	3 780	276	17	25	17	12 174	1524	1524 (0)

(C(K) enum column is the sum of the two scans.) Remaining ops (E0 build and generator search before the IC) are below 0.3% of the total at every q.

Fit: log2(cost) against log2(ℓ), OLS over q = 5 .. 23 (7 sizes)

series	slope	s.e.
IC total (`ops_total`)	2.495	0.032
φ(nG) table phase	2.513	0.034
C(K) enumeration	2.060	0.021
line-relation phase (q ≥ 5)	0.973	0.006
line relations + elimination row ops (q ≥ 7, 6 sizes)	1.006	0.014
number of line relations (q ≥ 7)	0.810	0.025
matched rho, $\sqrt{(\pi\ell/4)} \times 116$	0.500	0 (formula)
rho, measured plain-rho steps × 112	0.505	0.055

Reference slopes: Diem 4/9 = 0.444; rho 1/2. The measured IC slope is 2.50 against 0.444. The standard errors are the OLS errors of 5-7 points and say nothing about the validity of a power law over this range.

What the counted cost is made of (this is the main finding)

- The `ic` recovery in `quartic_ic.rs` does not solve the relation system: the rows are built and then discarded (`let _ = rows;`). The scalar comes from a table keyed by the degree-6 transfer divisor `φ(nG)` for every `n = 1..ℓ-1` (class_log), into which the three target divisors `φ(R0), φ(R1), φ(R2)` are looked up. That is an O(ℓ)-transfer enumeration of the subgroup, with per-transfer cubic reduction; it is not index calculus on Jac(C), and its measured slope (2.5) is that of this enumeration, not of Diem's algorithm.
- The $O(q^6) = O(\ell^2)$ scan of C(K) (slope 2.06) is charged at 9 per point only, and runs twice.
- Only the line-relation phase and its elimination are index-calculus work. They are about 10^{-6} to 10^{-7} of the total at q = 17-23.

- The relation phase scales as ≈ ℓ^1 (q^3): the tool enumerates the points of each line by scan (`line_points`), O(q) per pair, against the O(1) per pair Diem's count assumes. Relations found: 4 of 15 columns at q = 13 and 17 of 25 at q = 23 (grows as ℓ^{0.81}, about q^{2.4}); the rank equals the number of relations at every q, and rank < columns at all tested q, so no q here has a determined system (see below).

(a), (b), (c)

(a) Degree-3 canonical forms. Cubic reduction (`pic_canon`) of the valid `φ(nG)` gives a single degree-3 orbit for most of them (`results/run-20261018-canon3-collisions-q5-13.jsonl`):

q	valid φ(nG)	single deg-3 orbit	distinct orbits	collisions
5	24	23	21	2
7	44	41	34	7
11	164	141	121	20
13	296	244	196	48

Distinct n share a degree-3 form, so there is no unique degree-3 canonical form from this reduction, as the README says. Self-check run: the degree-6 keys are pairwise distinct for all valid n = 1..ℓ-1 with a degree-6 transfer (`n_deg6_key_collisions` = 0 at every q 5-23). That is a key-distinctness check, not a class-distinctness proof. Collisions did not block any size, because the recovery uses degree-6 keys. No canonical form was implemented, none patched silently; measuring collisions at q ≥ 17 was not done (counter added after the q = 17, 19, 23 runs began).

(b) The factor-base-only line system does not determine the ℓ-part: columns exceed rank at all seven q (rank = relations). The relation count grows faster than the column count (ratio 0.27 ⟶ 0.68 for q = 13 ⟶ 23), so the count may cross the column count near q ≈ 25-30 (extrapolation of seven points, not measured); even then the README's remark stands that the λ-projection of C(F_q) differences is often 4-primary, which was not re-tested here (no Jac(C) class arithmetic outside the tool's checks).

(c) Cost scaling: as above; the PROTOCOL's own prediction P2 (ratio rising with q) holds on these points (IC / rho from 6.3e3 to 3.7e7), for a reason unrelated to the dense elimination.

Why q = 29, 31 were not run

Measured wall (CPU) for one trial of `full`: q = 13 about 0.7 min, q = 17 11.4 min (4.9 CPU), q = 19 26 min (13.6 CPU), q = 23 69.5 min (54.8 CPU, 4 cores shared). The tool's φ-table phase grows as ℓ^2.5; extrapolating the fit, q = 29 is about 8 times and q = 31 about 14 times the q = 23 CPU time (hours each, one core). Not run; nothing measured for them beyond the instance and the rho arm.

Limits

Host shared, no isolation tiers, no A/A or positive controls; wall times are only context. 10 targets for q ≤ 13 but 1 for q = 17-23 (ops are the same in every target at each q, as shown for q = 5-13; for q ≥ 17 this is assumed). One instance per q. Rho is priced by formula (and by plain-rho step counts), not by an ecbench arm, so the tool's table-lookup rho is not charged and the unpriced-work band of AGENTS.md §5a (0 / 0.1 / 1 GAE) was not applied. These are counted-work diagnostics in the PROTOCOL's pinned units; the PROTOCOL's "cannot earn below-rho" rule applies. No sub-rho claim; IC costs 10^3-10^7 times the matched rho here. Per the success rule this is the baseline distance, not an improvement.

Still owed

1. A solve of the relation system (rank mod ℓ with an ℓ-part generator), which needs the λ-projection of point differences and an index-calculus target descent in place of the $\varphi(nG)$ table.
2. Line points by O(1) per pair (as in the count) instead of an O(q) scan.
3. A unique degree-3 (or degree-g) Pic canonical form with a class-level self-check.
4. #Jac(C)(F_q) recomputed from point counts over F_q, F_{q^2}, F_{q^3}.
5. ecbench-registered arms, replay, isolation, and the unpriced-work band.
6. Collisions and cost at q = 17-23 for degree-3 forms; $q \geq 29$.